

Guia

Índice

1. Filosofia do Projeto (De Script para Projeto) 
2. Arquitetura do Projeto (O Mapa) 
 - ♦ 2.1. Estrutura de Pastas 
 - ♦ 2.2. O Porquê de Cada Pasta e Ficheiro-Chave 
3. Guia de Instalação e Configuração 
 - ♦ 3.1. Pré-requisito: Git 
 - ♦ 3.2. Configuração Inicial (Apenas Uma Vez) 
 - ♦ 3.3. Resolvendo Erros Comuns do Windows 
4. Padrões de Código (Obrigatório) 
 - ♦ 4.1. Padrão de Nomenclatura (Java-Style) 
 - ♦ 4.2. Regras do Template de Envio (GlpiSender) 
5. O Processo de Trabalho (GitFlow) 
 - ♦ 5.1. Nomenclatura de Branches 
 - ♦ 5.2. O Ritual de Trabalho Diário (Obrigatório) 
6. Definição das Tarefas (Features da Equipa) 
 - ♦ 6.1. Erik: Task_Retriever 
 - ♦ 6.2. Duda: Task_Parser 
 - ♦ 6.3. Pedro: Action_Executor 
 - ♦ 6.4. Breno: Task_Closer 
 - ♦ 6.5. Duda: Main_Orchestrator 



1. Filosofia do Projeto

A primeira pergunta a ser feita é: "Porquê tanta pasta, se o código cabia num ficheiro só?"

A resposta é que passamos de um script para um projeto.

- ♦ Um script é feito para ser executado e resolver um problema imediato.
- ♦ Um projeto é feito para ser mantido, atualizado e melhorado ao longo do tempo, por múltiplas pessoas.

Adotámos esta arquitetura por 4 motivos principais:

1. **Segurança** 🔒 : NUNCA enviamos segredos (tokens, senhas, URLs) para o GitHub. Eles ficam isolados num ficheiro `.env` que é ignorado.
2. **Manutenção** 🛠️ : Quando a API do GLPI mudar, saberemos exatamente qual ficheiro corrigir (ex: `src/auth/session.py`) sem ter de ler 800 linhas de código.

- 3. Organização** 📁 : Cada pasta tem uma única responsabilidade (Separação de Responsabilidades). `src/` guarda as "ferramentas", `scripts/` "usa" as ferramentas.
- 4. Trabalho em Equipe** 👥 : Esta estrutura permite que 4 pessoas trabalhem em partes diferentes (um no login, outro na leitura de dados) ao mesmo tempo, sem que um apague o trabalho do outro.



2. Arquitetura do Projeto

2.1. Estrutura de Pastas

Esta é a "planta baixa" do nosso projeto.

```
ITXAutoNTI/
├── .git/                ← (O "cérebro" do Git, fica oculto)
├── .github/             ← (Ficheiros de automação do GitHub, ex: Issues)
├── .gitignore           ← O "Porteiro" do Git
├── config/
│   ├── .env             ← O "Cofre" com os segredos (IGNORADO PELO GIT)
│   └── .env.example     ← O "Molde" do cofre (VAI PARA O GIT)
├── docs/                ← (Manuais, diagramas, documentação)
├── scripts/
│   └── main.py          ← O ficheiro que NÓS EXECUTAMOS
├── src/
│   ├── auth/            ← Gaveta da Autenticação
│   │   └── session.py
│   ├── data/            ← Gaveta de Leitura/Escrita de Dados
│   │   ├── reader.py
│   │   └── sender.py
│   ├── logic/           ← Gaveta da Lógica
│   │   ├── task_parser.py
│   │   ├── action_executor.py
│   │   └── task_closer.py
│   ├── utils/           ← Gaveta de "Ajudantes"
│   │   └── config_loader.py
├── tests/               ← (A nossa "Rede de Segurança" para testes)
├── requirements.txt     ← As bibliotecas necessárias - se adicionar algo comunique no
├── DD
└── venv/                ← A "Bolha" de bibliotecas (IGNORADA PELO GIT)
```

2.2. O Porquê de Cada Pasta e Ficheiro-Chave

`src/` (Source/Fonte)

- ♦ **O que é?** É o "coração" do nosso projeto. Contém todo o nosso código Python que pode ser reutilizado. São as nossas "ferramentas".

- ◆ **Porquê? Separamos o código principal (`src/`) dos scripts que o executam (`scripts/`).**
- ◆ **`src/auth/` : Gaveta só para coisas de login.**
- ◆ **`src/data/` : Gaveta só para ler e escrever dados no GLPI.**
- ◆ **`src/logic/` : Gaveta para as regras de negócio e orquestração de tarefas.**
- ◆ **`src/utils/` : Gaveta para "ajudantes" que outros módulos usam.**

`scripts/`

- ◆ **O que é? Contém os ficheiros que nós executamos no terminal (ex: `python scripts/main.py`).**
- ◆ **Porquê? Estes scripts são os "trabalhadores". Eles vão até a `src/` (a "caixa de ferramentas"), pegam as ferramentas que precisam (como `autenticar_glpi`) e usam-nas para fazer um trabalho.**

`config/`

- ◆ **O que é? O nosso "cofre".**
- ◆ **`config/.env` : O ficheiro secreto. Contém os tokens e senhas reais. É IGNORADO PELO GIT.**
- ◆ **`config/.env.example` : O "molde" ou "exemplo". É um ficheiro que VAI PARA O GIT e mostra aos outros programadores quais variáveis eles precisam de criar no `.env` deles.**

`src/utils/config_loader.py`

- ◆ **O que é? É o "Chaveiro".**
- ◆ **Porquê? É o único script que sabe onde o "cofre" (`config/.env`) está. Ele abre o cofre e carrega os segredos para a memória do programa, para que o resto do código (como `session.py`) os possa usar em segurança, sem nunca saber quais são os tokens reais.**

`.gitignore`

- ◆ **O que é? O "Porteiro" do Git.**
- ◆ **Porquê? É uma lista de ficheiros e pastas que o Git deve IGNORAR. Os mais importantes que estão lá:**
 - ◆ `venv/` (Para não enviar 10.000 ficheiros de bibliotecas)
 - ◆ `config/.env` (Para não enviar os nossos segredos)
 - ◆ `__pycache__/` (Ficheiros temporários do Python)

`requirements.txt`

- ◆ **O que é? A "Lista de Compras" do projeto.**
- ◆ **Porquê? Lista todas as bibliotecas (`httpx` , `python-dotenv`) que o projeto precisa. Quando um colega novo entra, ele não precisa de adivinhar: apenas roda `pip install -r requirements.txt` e o seu `venv` é preenchido com tudo o que é necessário.**

`venv/`

- ◆ **O que é? O nosso "Ambiente Virtual" ou "Bolha".**
- ◆ **Porquê? Quando instalamos uma biblioteca (ex: `httpx`), ela é instalada *dentro* desta bolha, e não no seu Windows. Isso impede que os projetos interfiram uns com os outros. É IGNORADO PELO GIT.**

3. Guia de Instalação e Configuração

3.1. Pré-requisito: Git

Antes de começar, é **obrigatório ter o Git instalado no seu computador**. Todo o nosso fluxo de trabalho, incluindo os comandos de terminal para versionamento (como `git checkout`, `git pull`, `git commit`), depende dele para funcionar.

3.2. Configuração Inicial (Apenas Uma Vez)

Sempre que começar a trabalhar numa máquina nova (ou clonar o projeto), use o PowerShell:

PowerShell

```
# 1. Cria a "bolha" venv
python -m venv venv

# 2. Ativa a "bolha"
.\venv\Scripts\Activate.ps1

# 3. Instala a "lista de compras" dentro da bolha
pip install -r requirements.txt

# 4. Copia o "molde" dos segredos
copy config\.env.example config\.env

# 5. ABRA O FICHEIRO 'config/.env' E PREENCHA COM OS SEGREDOS REAIS
```

3.3. Resolvendo Erros Comuns do Windows

! Alerta Comum: Se ao tentar ativar o `venv` (`.\venv\Scripts\Activate.ps1`) der um erro vermelho sobre "execução de scripts foi desabilitada", rode este comando uma vez para permitir:

PowerShell

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
```

...e então tente ativar novamente (`.\venv\Scripts\Activate.ps1`).



4. Padrões de Código (Obrigatório)

Para manter a consistência e a legibilidade do projeto, todos os contribuidores **DEVEM** seguir os padrões abaixo.

4.1. Padrão de Nomenclatura (Java-Style)

Esta é a regra mais importante para a consistência do nosso código.

Regra Principal: Embora o programa seja em Python, toda a nomenclatura de código (variáveis, funções e classes) **DEVE** seguir as convenções do Java.

1. Nomes de Ficheiros (`.py`)

- ◆ **Regra:** Ficheiros devem usar `snake_case` (tudo minúsculo, separado por underscore).
- ◆ **Porquê:** Este é o padrão do Python. Usar outro formato (como `PascalCase`) quebra a consistência do ecossistema e pode complicar os `imports`.
- ◆ **Exemplos:**
 - ◆ `task_retriever.py`
 - ◆ `action_executor.py`

2. Nomes de Variáveis

- ◆ **Regra:** Devem usar `camelCase` (começa com minúscula, palavras seguintes com maiúscula).
- ◆ **Exemplos:**
 - ◆ `sessionToken`
 - ◆ `apiUrl`
 - ◆ `dailyTasksList`

3. Nomes de Funções e Métodos

- ◆ **Regra:** Devem usar `camelCase` (igual às variáveis).
- ◆ **Exemplos:**
 - ◆ `def getDailyTasks(robotUserId):`
 - ◆ `def parseInstruction(rawMessage):`
 - ◆ `def executeAction(instruction):`

4. Nomes de Classes

- ◆ **Regra:** Devem usar `PascalCase` (começa com maiúscula).
- ◆ **Exemplos:**
 - ◆ `class TaskRetriever:`
 - ◆ `class GlpiSender:`

5. Nomes de Constantes

- ◆ **Regra:** Devem usar `UPPER_SNAKE_CASE` (tudo maiúsculo).
- ◆ **Exemplos:**
 - ◆ `ROBOT_USER_ID = 39160`
 - ◆ `API_URL = settings.get("GLPI_API_URL")`

4.2. Regras do Template de Envio (GlpiSender)

O template `src/data/sender.py` (que já criámos) é a ferramenta oficial para atualizar ou criar qualquer item no GLPI.

- ◆ **Regra 1 (Injeção):** O `GlpiSender` nunca deve autenticar-se. Ele deve receber um cliente `httpx` já autenticado (que virá do `Authenticator` ou do `main`).

- ♦ **Regra 2 (Formato GLPI):** Todos os *payloads* (dados) enviados para a API devem seguir o formato `{"input": { ... }}`. O `GlpiSender` deve ser responsável por adicionar este "envelope" automaticamente.
- ♦ **Regra 3 (Métodos):**
 - ♦ Usar `createItem()` (método `POST`) para criar novos itens (ex: `ITILFollowup` num chamado).
 - ♦ Usar `updateItem()` (método `PUT`) para modificar itens existentes (ex: dar "check" numa tarefa, atualizar um campo de um ticket).
- ♦ **Regra 4 (Retorno):** Toda função no `GlpiSender` deve `return` um objeto ou `dict` de resultado (ex: `{"success": true, "message": " ... ", "data": ... }`), para que o Orquestrador saiba o que aconteceu.



5. O Processo de Trabalho (GitFlow)

Para que esta comunicação funcione, devemos seguir o fluxo GitFlow Simplificado. Este processo é gerido através de comandos Git no terminal.

- ♦ **Branches Protegidas:** `main` e `develop`.
- ♦ `main`: Versão estável. Ninguém toca.
- ♦ `develop`: É a nossa branch de "ponto de encontro". Ela representa a unificação de todas as funções (features) que já foram aprovadas e testadas como funcionais.
- ♦ `feature/ ...` (Branches de Tarefa): São as branches com os nomes das tarefas (ex: `feature/task-retriever`). Cada programador é totalmente responsável pela sua própria branch de feature. Isso inclui o controle de versionamento (fazer `commits` e `push` regulares) e garantir que o seu código funciona antes de pedir a integração (Pull Request).

5.1. Nomenclatura de Branches

- ♦ **Regra:** As branches de trabalho DEVEM seguir o padrão `feature/nome-da-tarefa`.
- ♦ **Exemplos:**
 - ♦ `feature/task-retriever`
 - ♦ `feature/task-parser`
 - ♦ `feature/action-executor`
 - ♦ `feature/task-closer`
 - ♦ `feature/main-orchestrator`

5.2. O Ritual de Trabalho Diário (Obrigatório)

Este é o fluxo que CADA membro da equipa deve fazer SEMPRE que for começar a trabalhar.

1. Ativar o Ambiente (Início do dia):

PowerShell

```
# 1. Ativa a "bolha" (se ainda não estiver ativa)
.\venv\Scripts\Activate.ps1
# (O seu prompt deve agora mostrar (venv) no início)
```

2. Sincronizar (Sempre!):

PowerShell

```
# 1. Vá para o "ponto de encontro"
git checkout develop

# 2. Receba as últimas atualizações dos seus colegas
git pull origin develop
```

3. Criar a sua Branch (Se for a primeira vez na tarefa):

PowerShell

```
# 3. Crie a sua "cópia de trabalho" pessoal a partir da 'develop' atualizada
git checkout -b feature/minha-tarefa
# (ex: git checkout -b feature/task-retriever)
```

(Se já tem a branch, apenas entre nela: `git checkout feature/minha-tarefa`)

4. Trabalhar (O dia-a-dia):

PowerShell

```
# --- (Você faz o seu código no seu ficheiro: src/data/task_retriever.py) ---
# ...
# --- (Trabalho concluído por hoje) ---
```

5. Salvar e Partilhar (O fim do dia):

PowerShell

```
# 1. Veja o que mudou
git status

# 2. Adicione os seus ficheiros
git add .

# 3. Crie um "save" (commit)
git commit -m "feat(retriever): Adiciona filtro por data nas tarefas"

# 4. Envie o seu "save" para o GitHub (na sua branch)
git push origin feature/minha-tarefa
```

6. O Pull Request (PR) (Quando a tarefa estiver PRONTA):

1. Vá ao GitHub.

2. Abra um Pull Request (PR) da sua branch (ex: `feature/task-retriever`) para a branch `develop`. **SE TEU CODIGO ESTIVER FUNCIONANDO**

3. Marque Duda como "Revisores" (Reviewers).

4. O seu código NÃO entra no projeto até ser revisto e aprovado.

5. É aqui que o trabalho em equipa acontece!

6. Definição das Tarefas (Features da Equipe)

Cada membro da equipe focará num módulo (`.py`) dentro da pasta `src/` . Duda (Main_Orchestrator) será responsável por importar e "chamar" estes módulos.

6.1. Erik: Task_Retriever

Conceito:

O seu trabalho é encontrar e organizar o sistema de tarefas que o ITXAutoNTI lerá e assim entregar de forma limpa para o Task_Parser.

- ◆ **Ficheiro:** `src/data/task_retriever.py`
- ◆ **Classe/Função Principal:** `def getDailyTasks(robotUserId):`
- ◆ **Missão:**
 - ◆ **Leitura do arquivo Guia.mb**
 - ◆ **Pensar em como a tarefa deve ser padronizada:**
 - ◆ Considerar a mensagem a melhor forma user friendly mas também que você consiga enviar para o Task_Parser;
 - ◆ Considerar o planejamento de execução pelo GLPI, levando em conta dias e horas.
 - ◆ Definir filtros de chamados (status, grupo responsável, etc.).
 - ◆ Estabelecer a frequência e rotina de execução do código.
 - ◆ Tratar tarefas duplicadas e o controle de checkbox (qualquer usuário pode dar checkout tarefas, se tiver como travar isso para ele seria bom, tipo se tiver algo no GLPI que trave alguém de dar check nas tarefas dele).
 - ◆ **Buscar todas as tarefas disponíveis.**
 - ◆ **Retornar uma lista de objetos estruturados, contendo:**
 - ◆ `taskId` → ID da tarefa (se existir)
 - ◆ `parentTicketId` → ID do chamado principal onde a tarefa está
 - ◆ `rawMessage` → Texto da tarefa que será interpretado pelo Task_Parser
 - ◆ Considerar possíveis coisas que podem ter, pensando que é de interesse que também seja atualizado o status e localização
 - ◆ Separar mensagem da forma mais eficiente para o Task_Parser (apenas que decida como vai enviar, se é por matriz, por vetor, por string, csv, pensar nisso)
 - ◆ `errorMessage` → Mensagem de erro caso haja alguma incompatibilidade ou problema que impeça a execução

6.2. Duda: Task_Parser

Conceito:

Responsável por interpretar o texto (string ou tabela) recebido do Task_Retriever, e convertê-lo em um objeto estruturado que o Action_Executor possa executar.

- ◆ **Ficheiro:** `src/logic/task_parser.py`
- ◆ **Classe/Função Principal:** `def parseTaskInstruction(rawMessage):`
- ◆ **Missão:**
 - ◆ **Leitura do arquivo Guia.mb**

- ◆ Receber a `rawMessage` (indefinido) que o `Task_Retriever` encaminhou.
- ◆ Analisar o padrão da mensagem (formato, palavras-chave, estrutura de dados).
- ◆ Extrair as instruções de ação — inicialmente inserção de patrimônios, mas considerar futura expansão para remoção e atualização.
- ◆ Identificar e separar IDs de patrimônio e demais informações associadas.
- ◆ Verificar se os IDs existem no GLPI e qual a classificação de cada um.
- ◆ Retornar um objeto `Instruction` estruturado com os seguintes campos:
 - ◆ `actionType` → Tipo de ação (por enquanto “inserir”)
 - ◆ `itemID` → Prefixo + ID do patrimônio
 - ◆ `itemType` → Classificação do item no GLPI
 - ◆ `itemLoc` → Localização informada na mensagem
 - ◆ `itemStatus` → Status descrito

6.3. Pedro: `Action_Executor`

Conceito:

Executa as instruções estruturadas vindas do `Task_Parser`, realizando inserções e atualizações dentro do sistema GLPI.

- ◆ Ficheiro: `src/logic/action_executor.py`
- ◆ Classe/Função Principal: `def executeInstruction(instruction):`
- ◆ Missão:
 - ◆ Leitura do arquivo `Guia.mb`
 - ◆ Receber o objeto `Instruction` do `Task_Parser` + `parentTicketId` do `Task_Retriever` ;
 - ◆ Acessar o chamado correspondente e inserir os patrimônios indicados:
 - ◆ `Glpisender` → Para inserir/atualizar dados no chamado.
 - ◆ `Glpireader` → Igual o `Task_Parser` vai fazer, só que no teu caso você vai alterar, ele só lê, você vai ler e alterar
 - ◆ Exemplo: o `Task_Parser` vai falar que o Status atual está X, e a tarefa vai pedir que o sejam alterados alguns detalhes, então as vezes vai pedir para alterar o status para Y. Isso deve vir do `Task_Parser` pra você, já pensa nisso por favor;
 - ◆ Implementar a lógica de inserção, mas já deixar preparado para futuras ações como atualizar e remover.
 - ◆ Retornar um objeto `Result` estruturado, contendo:
 - ◆ `Result` → Gera um detalhamento do que foi solicitado e como foi o status se foi inserido ou não.
 - ◆ Tu pensa em como vai entregar isso pro `Task_Closer`, ele precisa informar detalhamento através de uma mensagem geral no chamado e um arquivo em anexo para deixar claro o que foi feito.

6.4. Breno: `Task_Closer`

Conceito:

Finaliza tarefas e gera relatórios automáticos após a execução das ações.

- ◆ Ficheiro: `src/logic/task_closer.py`
- ◆ Classe/Função Principal: `def closeAndReportTask(taskToClose, result):`
- ◆ Missão:

- ♦ **Leitura do arquivo Guia.mb**
- ♦ **Receber o `taskId` original do Task_Retriever e o `result` retornado pelo Action_Executor.**
- ♦ **Gerar relatório CSV:**
 - ♦ **Criar a função `createCsvReport(result)` - já tenho essa função pronta, se quiser te envio, mas se quiser fazer sozinho não tem problema** - que gera um arquivo `.CSV` com o resumo das ações realizadas (ex: "id_identificado", "status de lançamento").
 - ♦ **id_identificado** → é a primeira coluna, se refere ao que foi identificado e salvo em `itemID`
 - ♦ **status de lançamento** → é a segunda coluna, refere ao que foi dado pelo Action_Executor, `Result`
- ♦ **Atualizar o GLPI usando o GlpiSender com três ações:**
 - 1. Dar check na tarefa:**
 - ♦ **Pensar em como vai ficar tarefas que foi feita a execução, mas por exemplo os patrimonios não puderam ser inseridos por não existir ou por motivo X.**
 - 2. Lançar mensagem de resultado:**
 - ♦ **Garantir que erros de upload ou de atualização sejam tratados com mensagens claras.**
 - 3. Anexar o CSV na mensagem que será lançada**
 - ♦ **Certificar-se de que o `.CSV` gerado tem estrutura limpa e padronizada.**

6.5. Duda: `Main_Orchestrator`

- ♦ **Ficheiro:** `scripts/main.py`
- ♦ **Missão:** Escrever o `main()` que "amarra" tudo. Python

```
def main():
    # 1. Chamar Autenticação
    # 2. Chamar getDailyTasks(robotUserId)
    # 3. Loop `for task in listaDeTarefas:`
    # 4.     Chamar Duda(Parser).parseTaskInstruction(task.rawMessage) → obtém
    `instrucao`
    # 5.     Chamar Pedro.executeInstruction(instrucao) → obtém `resultado`
    # 6.     Chamar Breno.closeAndReportTask(task, resultado)
    # 7. Fim do Loop
    # 8. Deslogar
```